

Chapter 17: Polymorphism in Java

Personal Notes

Polymorphism is one of the four pillars of OOP. The word comes from Greek and literally means "many forms." In Java, it means a single method, object, or interface can behave in different ways depending on the situation.

Polymorphism is what makes Java code flexible. The same method call can do different things depending on what kind of object is behind it. This is the trick that lets frameworks, libraries, and large applications stay clean and extensible.

1. What is Polymorphism?

In simple terms, polymorphism means one thing behaving differently in different contexts.

Real-life examples:

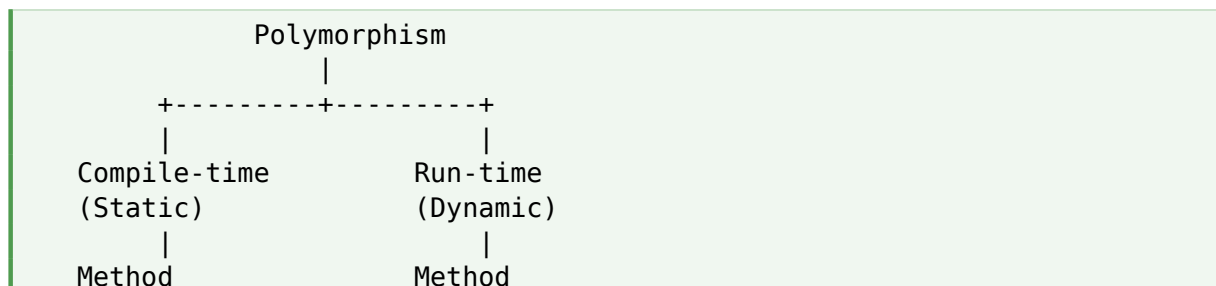
- The word "draw" — a painter draws on canvas, an ATM draws money, a gunfighter draws a weapon. Same word, very different actions.
- A person can be a son, a student, an employee, a friend — same person, different behaviors depending on context.
- A + button — adds two numbers in a calculator, joins two strings in a chat app. Same operator, different meaning based on the data.

In Java code, polymorphism shows up as the same method name behaving differently based either on the arguments passed in, or on the actual object type at runtime.

2. Types of Polymorphism in Java

Java supports two main types of polymorphism:

- Compile-time Polymorphism (also called Static Polymorphism) — achieved through Method Overloading
- Run-time Polymorphism (also called Dynamic Polymorphism) — achieved through Method Overriding



Overloading	Overriding
Decided at compile time by the compiler	Decided at runtime by the JVM

3. Compile-time Polymorphism (Method Overloading)

Compile-time polymorphism is resolved by the Java compiler BEFORE the program even runs. The compiler looks at the arguments you pass and picks which method version to call. In Java, this is achieved through Method Overloading.

3.1 What is Method Overloading?

Method overloading means defining multiple methods with the SAME name in the SAME class, but with different parameters. "Different" can mean:

- Different NUMBER of parameters
- Different TYPES of parameters
- Different ORDER of parameter types

3.2 Example

```
class Display {

    // Method 1 – takes an int
    void display(int a) {
        System.out.println("Integer: " + a);
    }

    // Method 2 – takes a String
    void display(String s) {
        System.out.println("String: " + s);
    }

    // Method 3 – takes two ints
    void display(int a, int b) {
        System.out.println("Two integers: " + a + ", " + b);
    }
}

public class Main {
    public static void main(String[] args) {
        Display d = new Display();
        d.display(10);
        d.display("Hello");
        d.display(5, 15);
    }
}
```

```
}
```

Output:

```
Integer: 10  
String: Hello  
Two integers: 5, 15
```

All three methods share the name `display`, but Java picks the right one based on the type and number of arguments. This decision happens at compile time — that's why it's called compile-time polymorphism.

Important: You CANNOT overload a method just by changing the return type. Java decides which version to call based on the arguments, not the return type. Two methods with the same name and same parameters but different return types will fail to compile.

4. Run-time Polymorphism (Method Overriding)

Run-time polymorphism is resolved during program EXECUTION, not at compile time. In Java, this is achieved through Method Overriding combined with Inheritance. This is where the real magic of polymorphism shows up.

4.1 What is Method Overriding?

Method overriding means a CHILD class defines a method with the EXACT same name and parameters as one in its parent class. When you call that method, Java decides at runtime which version to execute — based on the ACTUAL object, not the reference type.

4.2 Example

```
class Animal {  
    void sound() {  
        System.out.println("Some generic animal sound");  
    }  
}  
  
class Dog extends Animal {  
    @Override  
    void sound() {  
        System.out.println("Dog says: Woof!");  
    }  
}  
  
class Cat extends Animal {  
    @Override  
    void sound() {  
        System.out.println("Cat says: Meow!");  
    }  
}
```

```

public class Main {
    public static void main(String[] args) {
        Animal a;           // reference type is Animal

        a = new Dog();
        a.sound();           // calls Dog's sound() at runtime

        a = new Cat();
        a.sound();           // calls Cat's sound() at runtime
    }
}

```

Output:

```

Dog says: Woof!
Cat says: Meow!

```

Why this is amazing: The variable `a` is declared as `Animal` — but at runtime it holds a `Dog` object, then a `Cat` object. Java looks at the ACTUAL object at runtime and picks the right overridden method. The same line of code (`a.sound()`) produces totally different results depending on what's behind the reference.

4.3 Why It's Called Dynamic Dispatch

```

Reference type:  Animal  a
      |
      v
Actual object:   Dog (at this moment)
      |
      v
Java says: "What is the real object?"
          "A Dog → call Dog's sound()"

This decision happens at RUNTIME,
which is why it's called dynamic dispatch
(or late binding).

```

5. Overloading vs Overriding

These two are completely different concepts. Don't mix them up — interviewers love asking the difference.

Aspect	Overloading	Overriding
Where it happens	Within the SAME class	Across PARENT and CHILD classes
Method name	Same	Same
Parameters	MUST be different	MUST be exactly the same

Aspect	Overloading	Overriding
Return type	Can vary	Should match (or be a subtype)
Inheritance needed	No	Yes — requires extends
When resolved	Compile time	Runtime
Also called	Compile-time / Static polymorphism	Run-time / Dynamic polymorphism
Example	add(int,int) + add(double,double)	Dog overrides Animal's sound()

6. The @Override Annotation

When you override a method in a child class, it is a good practice to write `@Override` above it. This tells the Java compiler that you INTEND to override a parent method.

Why it matters

If you make a typo in the method name or get a parameter type wrong, the `@Override` annotation makes the compiler throw an error immediately. Without it, Java would silently treat your method as a brand new one — and the override would never happen. You'd be left wondering why the parent's version is still running.

Example

```
class Animal {
    void sound() {
        System.out.println("Generic animal sound");
    }
}

class Dog extends Animal {
    @Override
    void sound() {                // correctly overrides
        System.out.println("Woof!");
    }
}

class Cat extends Animal {
    @Override
    void soundd() {                // TYPO! @Override catches it
        System.out.println("Meow!"); // compiler ERROR — no parent method
    }                               // named "soundd" to override
}
```

Best practice: ALWAYS add `@Override` when overriding. It costs nothing to write and saves hours of "why isn't my method being called" debugging.

7. Abstract Classes and Abstract Methods

Polymorphism is often used together with abstract classes. An abstract class is one that CANNOT be instantiated directly — you can't create an object of it. It's only meant to be used as a parent for other classes.

An abstract method is a method that has NO body — no implementation. The parent class declares it, but the child classes MUST provide the actual code.

Example

```
abstract class Shape {
    // abstract method – no body
    abstract void draw();

    // regular method – has a body, inherited as-is
    void info() {
        System.out.println("This is a shape.");
    }
}

class Circle extends Shape {
    @Override
    void draw() {
        System.out.println("Drawing a circle");
    }
}

class Rectangle extends Shape {
    @Override
    void draw() {
        System.out.println("Drawing a rectangle");
    }
}

public class Main {
    public static void main(String[] args) {
        Shape s;

        s = new Circle();
        s.draw();           // Drawing a circle
        s.info();           // This is a shape.

        s = new Rectangle();
        s.draw();           // Drawing a rectangle

        // Shape sh = new Shape();    // ERROR – can't instantiate
    }
}
```

Output:

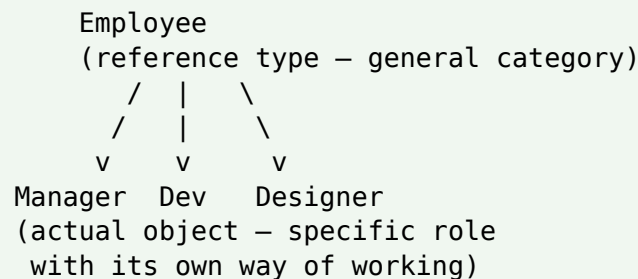
```
Drawing a circle
This is a shape.
Drawing a rectangle
```

Here, Shape is an abstract parent. Circle and Rectangle each provide their own version of draw(). Java picks the correct version at runtime — that's run-time polymorphism in action, supported by abstraction.

Rule: If a class has even ONE abstract method, the class itself must be declared abstract. And any class that extends an abstract class MUST implement all its abstract methods — or be abstract itself.

8. Real-World Analogy

Think of a person who works at a company. Their job title is "Employee." But that same person can play different specific roles — Manager, Developer, Designer, Tester.



Depending on the actual role, their work behavior changes:

- As a Developer, they write code
- As a Manager, they assign tasks and run meetings
- As a Designer, they create UI mockups

But to the world, they are all just "Employees" — the same general reference. That is exactly how polymorphism works in Java. The reference type is the general category, but the actual behavior at runtime depends on the specific object.

9. Advantages of Polymorphism

- Code Reusability — the same method name works for many types
- Flexibility — new child classes can be added without modifying existing code
- Reduced Redundancy — no need to write separate logic for each type
- Cleaner Design — code reads at a higher, more general level
- Easier Maintenance — change behavior in one place, ripples through the hierarchy
- Open/Closed Principle — your code stays "open for extension, closed for modification"

10. Common Mistakes

Mistake 1: Trying to overload by return type only

```
int add(int a, int b)    { return a + b; }  
double add(int a, int b) { return a + b; } // ERROR – same params
```

Java picks methods by ARGUMENTS, not return type. Two methods with the same parameters can't coexist.

Mistake 2: Overriding with different parameters

```
class Animal {  
    void sound() { ... }  
}  
  
class Dog extends Animal {  
    void sound(String s) { ... } // this is OVERLOADING, not overriding  
}
```

Parameters MUST match exactly for an override. Different parameters = it's a new method, not an override. @Override would catch this immediately.

Mistake 3: Forgetting @Override

Without @Override, a typo silently turns your override into a brand new method. Always use the annotation.

Mistake 4: Trying to instantiate an abstract class

```
Shape s = new Shape(); // ERROR – can't instantiate  
Shape s = new Circle(); // OK – instantiate a concrete subclass
```

Mistake 5: Confusing reference type and object type

In `Animal a = new Dog();` — the reference type is `Animal`, the actual object type is `Dog`. Polymorphism uses the OBJECT type to pick the method, not the reference type.

Mistake 6: Trying to access child-only methods through a parent reference

```
Animal a = new Dog();  
a.sound(); // OK – overridden method  
a.fetch(); // ERROR – Animal doesn't know about fetch()
```

If a method exists only in the child, you can't call it through a parent reference without casting.

11. Quick Reference

Term	Meaning
Polymorphism	One method/object behaves differently in different contexts

Term	Meaning
Compile-time poly.	Decided at compile time → Method Overloading
Run-time poly.	Decided at runtime → Method Overriding
Method Overloading	Same name + different parameters in same class
Method Overriding	Child redefines parent's method with same signature
@Override	Tells compiler this method IS an override — catches typos
abstract class	Cannot be instantiated; meant to be extended
abstract method	No body in parent; children must implement
Dynamic dispatch	Runtime decision of which overridden method to call
Reference type	Declared type of variable (Animal a = ...)
Object type	Actual class of the object (new Dog())

12. Practice Problems

Try these to lock in both kinds of polymorphism.

1. Create a Calculator class with overloaded methods: add(int, int), add(double, double), add(int, int, int).
2. Build a Shape hierarchy with abstract method area(). Implement it in Circle, Rectangle, and Triangle.
3. Create an Employee parent with calculateSalary(). Override it in Manager, Developer, and Intern with different formulas.
4. Make a Vehicle parent with start(). Override it in Car, Bike, and Truck to print different messages.
5. Build a Payment system: abstract class Payment with pay(). Create CreditCard, UPI, and Cash subclasses.
6. Create a method print() overloaded for int, double, String, and an int array.
7. Demonstrate dynamic dispatch: declare Animal[] arr = { new Dog(), new Cat(), new Cow() }; loop and call sound() on each.
8. Show why @Override matters: intentionally make a typo in an overridden method name with and without @Override.

13. Points to Remember

- Polymorphism means "many forms" — one interface, many behaviors

- Java has two types: compile-time (overloading) and run-time (overriding)
- Overloading happens in the same class; overriding happens across parent and child
- All non-static, non-final, non-private methods in Java are virtual by default — meaning overriding works automatically
- Always use `@Override` when overriding — it catches bugs the compiler would otherwise miss
- Abstract classes use the abstract keyword and cannot be instantiated directly
- Any class that extends an abstract class must implement all its abstract methods
- Java decides overridden methods based on the OBJECT type, not the reference type

14. Final Takeaway

Polymorphism is what gives Java code its flexibility. Without it, every type would need its own separate code path. With it, you can write one method that handles many types, and the right behavior is picked automatically.

Master these two ideas: overloading (decided at compile time by arguments) and overriding (decided at runtime by the actual object). Once you can spot the difference instantly, polymorphism stops being theory and becomes a tool you reach for naturally.

Key Takeaway: Polymorphism is one method, many behaviors. Overloading happens at compile time and is about ARGUMENTS — the compiler picks the version. Overriding happens at runtime and is about OBJECTS — the JVM picks the version. Combine overriding with abstraction, and you get the foundation of every modern Java framework.

15. What's Next?

Now that polymorphism is clear, the natural next OOP topics are:

- Interfaces — pure abstraction and multiple inheritance
- The `instanceof` keyword — checking the real type at runtime
- Type casting between parent and child references (upcasting and downcasting)
- The `Object` class and methods every Java class inherits (`toString`, `equals`, `hashCode`)
- Generic methods — polymorphism over types
- Design patterns — Strategy, Factory, Template Method — all built on polymorphism